

IF330 Pemrograman Web.

04 - PHP Object Oriented Programming

Fenina Adline Twince Tobing

I Putu Ananta Yogiswara

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK DAN INFORMATIKA
UNIVERSITAS MULTIMEDIA NUSANTARA
SEMESTER GANJIL TAHUN AJARAN 2022/2023

REVIEW



- PHP Predefined Variables
- PHP Predefined Functions
- PHP Form Submission
- HTTP Cookies
- `json_encode` & `json_decode`

TODAY's OUTLINE



- Object-Oriented Programming
- Classes & Objects
- Constructor & Destructor
- Encapsulation / Access Modifier
- Inheritance & Final keyword
- Constants
- Abstract Classes
- Interface
- Traits
- Static Properties & Methods
- Autoloading
- Namespaces

Object Oriented Programming

- **Object-oriented programming (OOP)** refers to the creation of reusable software object-types / classes that can be efficiently developed and easily incorporated into multiple programs.
- In OOP an object represent an entity in the real world (a student, a desk, a button, a file, a text input area, a loan, a web page, a shopping cart).
- An OOP program = a collection of objects that interact to solve a task/problem

Object Oriented Programming

- **Objects are self-contained**, with data and operations that pertain to them assembled into a single entity.
 - In procedural programming data and operations are separate (this methodology requires sending data to methods!)
- **Objects have:**
 - **Identity**; ex: 2 "OK" buttons, same attributes, separate handle vars
 - **State**: a set of attributes (aka member variables, properties, data fields) = properties or variable that relate to / describe the object, with their current values.
 - **Behavior**: a set of operations (aka methods) = actions or functions that the object can perform to modify itself - its state, or perform for some external effect/result.

Object Oriented Programming

- **Encapsulation** (aka data hiding) central in OOP
 - access to data within object is available only via the object's operation (known as the interface of the object)
 - internal aspects of objects are hidden
- Advantages:
 - objects can be used as black-boxes, if their interface is known
 - implementation of an interface can be changed without a cascading effect to other parts of the project (if the interface doesn't change)

Object Oriented Programming

- **Classes** are constructs that define objects of the same type.
A class is a template or blueprint that defines what an object's data and methods will be.
Objects of a class have:
 - Same operations, behaving the same way
 - Same attributes representing the same features, but values of those attributes (=state) can vary from object to object
- An object is an instance of a class.
(terms objects and instances are used interchangeably)
- Any number of instances of a class can be created

Why Use Classes and Objects?

- PHP is a primarily procedural language
- Small programs are easily written without adding any classes or objects
- Larger programs, however, become cluttered with so many disorganized functions
- Grouping related data and behavior into objects helps manage size and complexity

Class

A **class** is a template for objects, and an **object** is an instance of class.

A class is defined by using the **class** keyword

```
<?php

class Student {
    public $nama; Property

    function sayHello() {
        echo "Hello, my name is " . $this->nama;
    }
}
```

Method

Object

Classes are nothing without objects!

We can have multiple objects from a class. Each object has all the properties and methods defined in the class, but they will have different property values.

Objects of a class is created using the **new** keyword.

```
require_once('file_class.php');

$obj = new Student();
$obj->nama = "John Thor"; // set property
echo $obj->nama; // John Thor
$obj->sayHello(); // Hello, my name is John Thor

$obj2 = new Student();
$obj2->nama = "John Wick";
echo $obj2->nama; // John Wick
$obj2->sayHello(); // Hello, my name is John Wick
```

Constructor & Destructor

- A constructor allows you to initialize an object's properties upon a creation of the object.
- **__construct()** function will automatically called when you create an object from a class
- A destructor is called when the object is destructed or the script is stopped or exited.
- **__destruct()** function will automatically called at the end of the script.

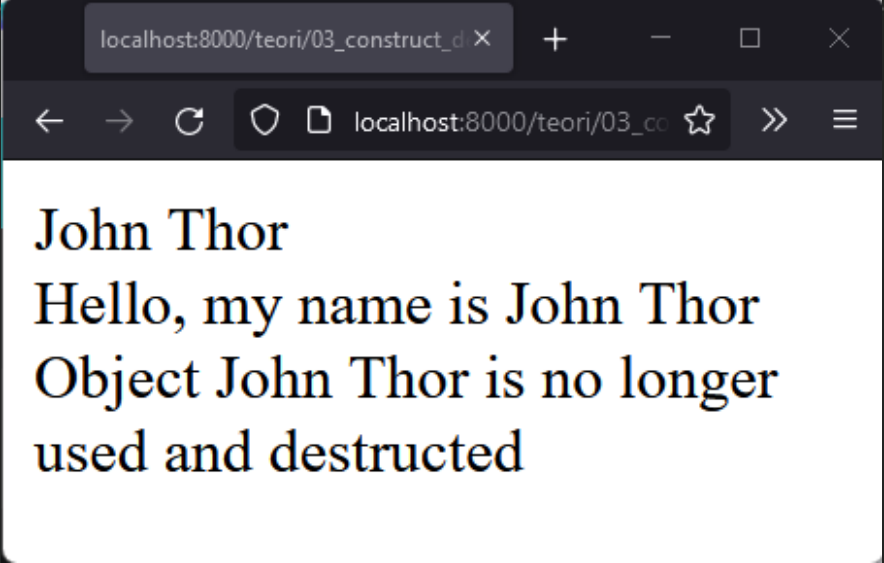
```
<?php
class Student {
    public $nama;

    function __construct($nama){
        $this->nama = $nama;
    }

    public function sayHello(){
        echo "Hello, my name is " . $this->nama;
    }

    function __destruct(){
        echo "Object $this->nama is no longer used and destructed";
    }
}

$obj = new Student( nama: "John Thor");
echo $obj->nama; echo "<br />";
$obj->sayHello(); echo "<br />";
```



Encapsulation

- Properties and methods can have access modifiers which control where they can be accessed.
- There are three access modifiers:
 - **public** - can be accessed from anywhere. (*default*)
 - **protected** - can be accessed within the class and by classes derived from that class
 - **private** - can only be accessed within the class

```
<?php
class Student{
    public $nim, $a, $b, $c;
    protected $nama;
    private $prodi;

    function set_a($a) {
        $this->a = $a;
    }

    protected function set_b($b) {
        $this->b = $b;
    }

    protected function set_c($c) {
        $this->c = $c;
    }
}
```

```
$obj = new Student;
$obj->nim = '001'; // OK
$obj->nama = 'John Thor'; // ERROR
$obj->prodi = 'Teknik InfoKomputer'; // ERROR

$obj->set_a(a: 'A'); // OK
$obj->set_b(b: 'B'); // ERROR
$obj->set_c(c: 'C'); // ERROR
```


Inheritance

- Inheritance in OOP = When a class derives from another class.
- The child class will inherit all the public and protected properties and methods from the parent class.
- An inherited class is defined by using the **extends** keyword.

```
class Kucing extends Hewan {  
    function berpindah(){  
        echo "Saya merangkak dengan 4 kaki";  
    }  
}
```

```
class Monyet extends Hewan { }
```

```
class Hewan {  
    protected $jml_kaki;  
  
    function berpindah(){  
        echo "Saya berpindah";  
    }  
  
    function makan() {  
        echo "Saya makan";  
    }  
}
```

```
class Burung extends Hewan {  
    protected $sayap;  
  
    function berpindah() {  
        echo "Saya terbang";  
    }  
  
    function makan() {  
        echo "Saya makan dengan mematuk";  
    }  
}
```

Constant

- Constants cannot be changed once its declared
- Constants can be useful if you need to define some constant data within a class
- Constant is declared inside a class with the const keyword
- Constants are case-sensitive.
- It's recommended to name the constants in all uppercase letters.

```
<?php
class Hello {
    const MESSAGE = "Hello World";
    public function salam(){
        echo self::MESSAGE;
    }
}

echo Hello::MESSAGE; // Hello World
$obj = new Hello();
$obj->salam(); // Hello World
```

Final

The **final** keyword can be used to prevent class inheritance or to prevent method overriding.

```
final class Mahasiswa { }
```

```
class MahasiswaIF extends Mahasiswa {  
    // ERROR can't extend final class  
}
```

```
class Student {  
    public function uts(){  
        echo self::ujian() . "Tengah Semester";  
    }  
  
    final public function ujian() {  
        echo "Ujian ";  
    }  
}
```

```
class StudentIF extends Student {  
    public function ujian(){  
        // ERROR: can't override final method  
        echo "Exam ";  
    }  
}
```


Abstract Classes

- Abstract classes and methods are when the parent class has a named method, but need its child(es) to fill out the tasks.
- An abstract class is a class that contains at least one abstract method.
- An abstract method is a method that is declared, but not implemented in the code.

```
abstract class AbstractClass {  
    // Force Extending Class to define this method  
    abstract protected function getValue();  
    abstract protected function prefixValue($prefix);  
  
    // Common method  
    public function printOut(){  
        print $this->getValue() . "\n";  
    }  
}
```

```
class ConcreteClass1 extends AbstractClass {  
    protected function getValue(){  
        return "ConcreteClass1";  
    }  
  
    public function prefixValue($prefix){  
        return "{$prefix}ConcreteClass1";  
    }  
}
```

```
class ConcreteClass2 extends AbstractClass {  
    public function getValue(){  
        return "ConcreteClass2";  
    }  
  
    public function prefixValue($prefix){  
        return "{$prefix}ConcreteClass2";  
    }  
}
```


Interfaces

- Interfaces allow you to specify what methods a class should implement.
- Interfaces make it easy to use a variety of different classes in the same way.
- When one or more classes use the same interface, it's referred to as **polymorphism**.
- Interfaces are declared with the **interface** keyword.

```
interface Animal {  
    public function makeSound();  
}  
  
class Cat implements Animal {  
    public function makeSound(){  
        echo "Meow...";  
    }  
}  
  
class Dog implements Animal {  
    public function makeSound() {  
        echo "Bark...";  
    }  
}
```

Interface vs Abstract Class

- Interfaces cannot have properties, while abstract classes can.
- All interface methods must be public, while abstract class methods is public or protected.
- All methods in an interface are abstract, so they cannot be implemented in code and the abstract keyword is not necessary.
- Classes can implement an interface while inheriting from another class at the same time.

Traits

- PHP only supports single inheritance: a child class can inherit only from one single parent.
- Traits are used to declare methods that can be used in multiple classes.
- Traits can have methods and abstract methods that can be used in multiple classes, and the methods can have any access modifier (public, private, or protected).
- Traits are declared with the **trait** keyword.

```
trait message1 {  
    public function msg1() {  
        echo "OOP is fun...";  
    }  
}  
  
trait message2 {  
    public function msg2() {  
        echo "message 2....";  
    }  
}  
  
class Welcome {  
    use message1, message2;  
}  
  
$obj = new Welcome();  
$obj->msg1();  
$obj->msg2();
```

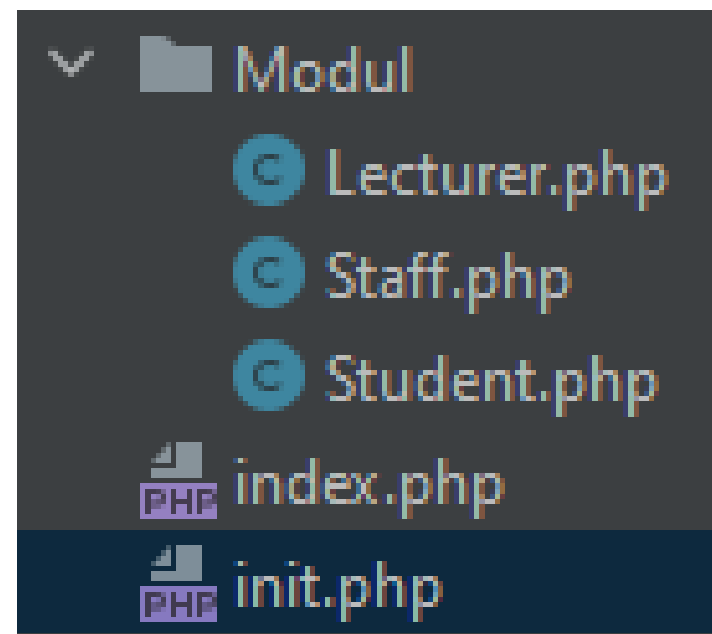
Static Properties & Methods

- Static properties and methods can be called directly - without creating an instance of the class first.
- static properties and methods are declared with the **static** keyword.
- A class can have both static and non-static properties and methods.
- A static properties and methods can be accessed from a method in the same class using the **self** keyword and double colon (::)

```
class Hello {  
    public static $pi = 3.14;  
    public static function welcome() {  
        echo "Hello World";  
    }  
}  
  
class Something {  
    public function message() {  
        Hello::welcome();  
    }  
}  
  
class Something2 extends Hello {  
    function __construct() {  
        parent::welcome();  
    }  
}
```

Autoloading

The **spl_autoload_register()** function registers any number of autoloaders, enabling for classes and interfaces to be automatically loaded if they are currently not defined. By registering autoloaders, PHP is given a last chance to load the class or interface before it fails with an error.

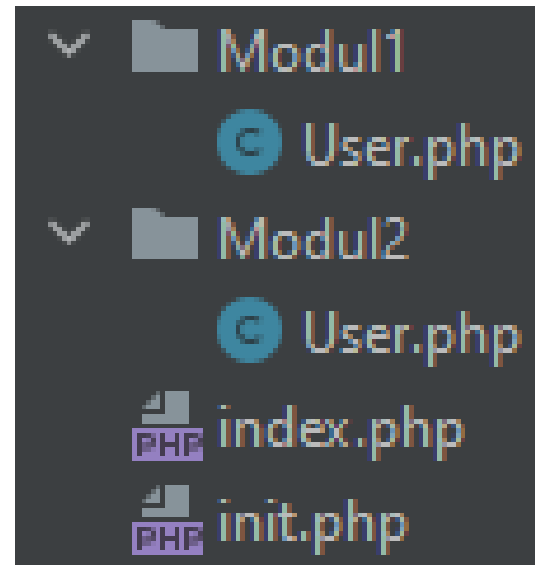


```
<?php
//require_once 'Modul/Student.php';
//require_once 'Modul/Staff.php';

// Load every class in the Modul dir
spl_autoload_register(function($class){
    require_once 'Modul/' . $class . '.php';
});
```


Namespaces

- Namespaces are used to organize a group of classes that work together to perform task.
- Namespaces allow the same name to be used for more than one class
- Namespaces are declared at the beginning of a file using the **namespace** keyword.
- The **use** keyword can be used to make an alias for a namespace or class inside a namespace.



```
namespace Modul1;

class User{
    function __construct(){
        echo "Ini adalah class " . __CLASS__;
    }
}
```

```
namespace Modul2;

class User{
    function __construct(){
        echo "Ini adalah class " . __CLASS__;
    }
}
```

```
// init.php
spl_autoload_register(function($class){
    $class = explode( separator: '\\', $class);
    $class = end( &array: $class);
    require_once 'Modul1/' . $class . '.php';
    require_once 'Modul2/' . $class . '.php';
});
```

```
require_once 'init.php';
use Modul1\User as User1;

new Modul1\User();
echo "<br />";
new Modul2\User();
echo "<br />";
new User1();
```

SUMMARY

After finishing this module, you should be able:

- to understand what is object-oriented programming
- to understand how to implement classes and objects in PHP
- to understand how to implement constructor and destructor in a class
- to understand how to implement polymorphism using access modifier
- to understand how to implement class inheritance
- to understand how to implement interfaces, abstract classes, traits, static properties and methods.
- to understand how to create autoloader
- to understand how to implement namespaces

NEXT WEEK'S OUTLINE

- File Upload
- Read & Write File
- Generate PDF & Excel File

Note: Please install **composer**

<https://getcomposer.org/>



REFERENCES

- <https://www.w3schools.com/PHP/default.asp>
- <https://www.php.net/docs.php>
- Luke Welling & Laura Thomson. 2017. PHP and MySQL Web Development (5th ed)
- Robin Nixon. 2021. Learning PHp, MySQL & JavaScript.

Visi

Menjadi Program Studi Strata Satu Informatika **unggulan** yang menghasilkan lulusan **berwawasan internasional** yang **kompeten** di bidang Ilmu Komputer (*Computer Science*), **berjiwa wirausaha** dan **berbudi pekerti luhur**.



Misi

1. Menyelenggarakan pembelajaran dengan teknologi dan kurikulum terbaik serta didukung tenaga pengajar profesional.
2. Melaksanakan kegiatan penelitian di bidang Informatika untuk memajukan ilmu dan teknologi Informatika.
3. Melaksanakan kegiatan pengabdian kepada masyarakat berbasis ilmu dan teknologi Informatika dalam rangka mengamalkan ilmu dan teknologi Informatika.